Course Notes

Lecture 06: Advanced BLE, Gateway, Client and Notifications

Communication Technologies for the Internet of Things

Academic year 2026/2027

Catarina Silva

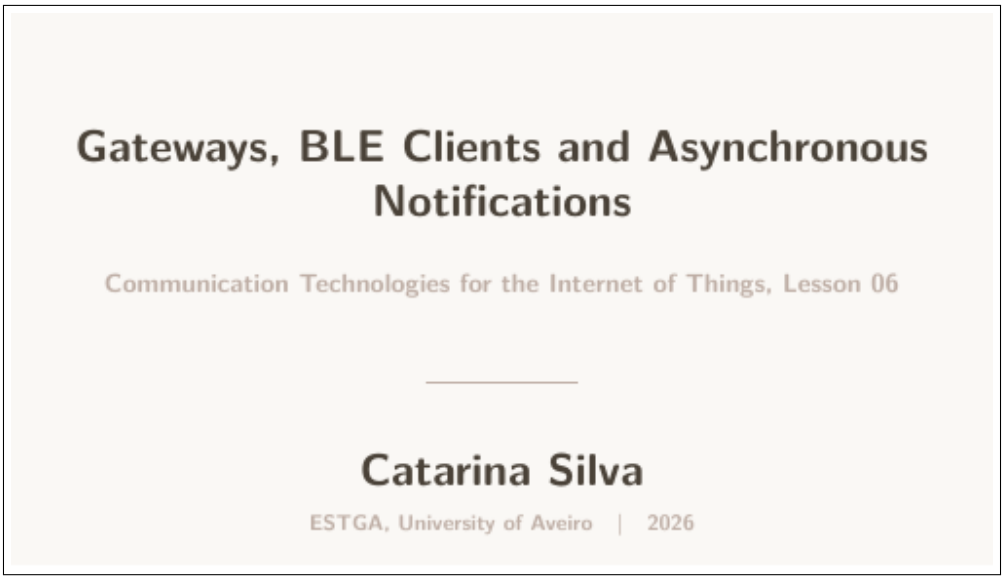
ESTGA, University of Aveiro

Text to support study

Contents

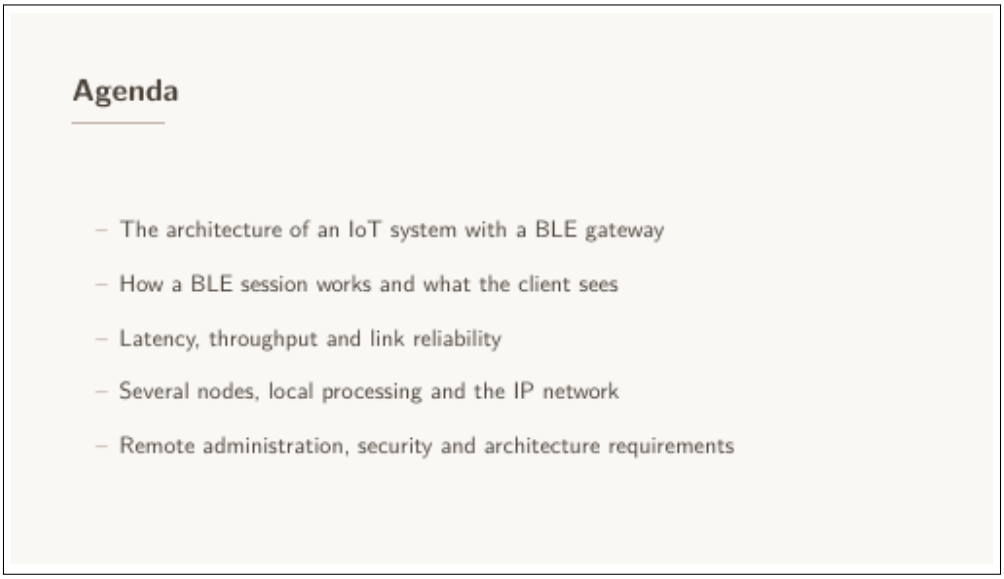
1. Title . . . 2
2. Agenda . . . 2
3. The gateway between two networks . . . 3
4. Layered architecture . . . 4
5. Roles and topology in BLE . . . 5
6. The BLE stack and where each layer runs . . . 6
7. Anatomy of a BLE session . . . 7
8. The GATT data model . . . 8
9. The Raspberry Pi as a BLE client: bluepy classes . . . 9
10. Polling and notification . . . 10
11. Where latency comes from . . . 11
12. Connection pace and trade-offs . . . 12
13. Throughput and fragmentation . . . 13
14. Connection loss and recovery . . . 14
15. Several nodes, one gateway . . . 15
16. Local processing and memory . . . 16
17. Forwarding to the IP network . . . 17
18. Gateway network questions . . . 18
19. Remote administration . . . 19
20. Security and privacy . . . 20
21. Architecture requirements and decisions . . . 21
22. Lecture summary . . . 22
23. Sources . . . 22
24. Closing . . . 23

1. Title



This lecture is the third one on Bluetooth Low Energy and it changes the point of view. In the earlier sessions the focus was on the device that advertises and answers. Now the focus is on the element that looks for it, connects to it, collects the data and delivers it to a data network. That element is called a gateway, and its design raises questions of architecture, networking, latency, scale and security, which the following text goes through.

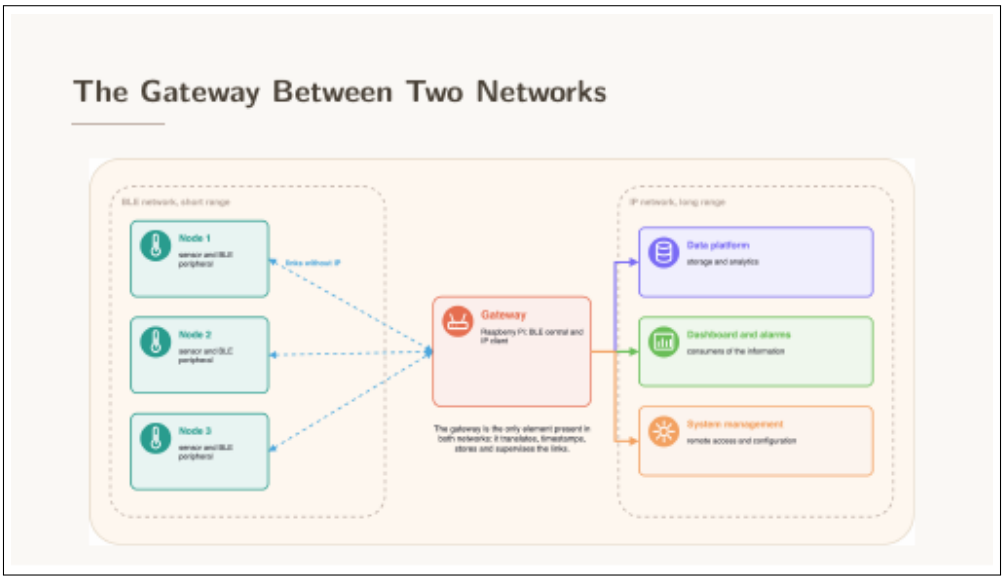
2. Agenda



The path starts with the gateway as an architectural element and with the place it occupies between networks. It goes on with the BLE mechanisms a gateway has to master: roles, layers, session and data model. It then treats the Raspberry Pi as a BLE client, the difference between polling and notification, and the causes of latency. The third part covers what happens when the system grows or fails: several nodes, loss of connection and local processing. It ends with

forwarding to the IP network, remote administration and security, and with a translation of requirements into decisions.

3. The gateway between two networks



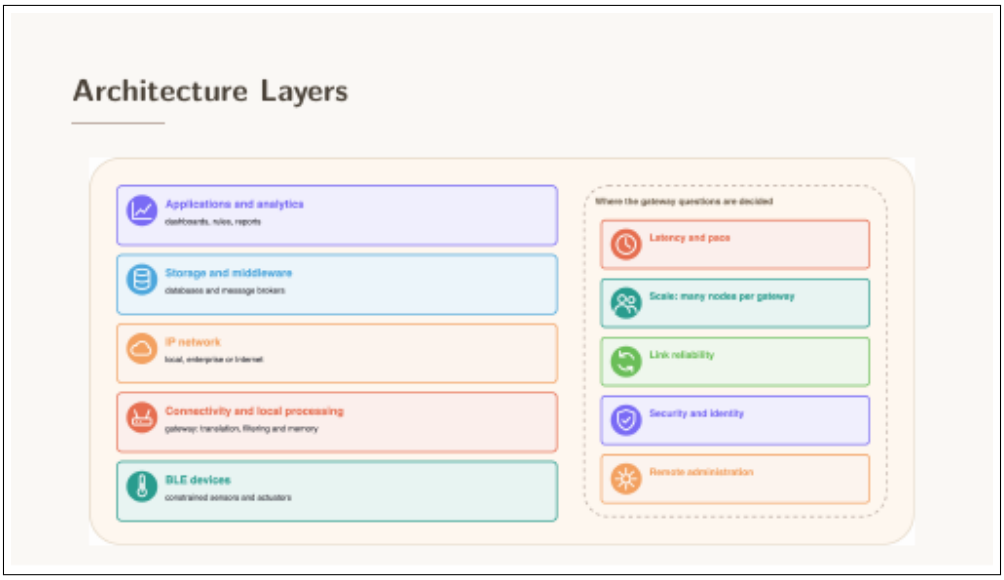
BLE was designed for short links between constrained devices. A BLE peripheral has no IP address, takes part in no data network and can only be reached by a device within range of its radio. The Internet and local data networks are built on IP. Between the two there is a gap that no sensor can cross alone.

The gateway crosses that gap because it has two interfaces. One is a BLE adapter, through which it acts as a central and keeps connections with the nodes. The other is an IP interface, through which it delivers the information to whoever consumes it. This position gives it a set of functions that the nodes cannot take on.

The first function is translation. The gateway receives values in attribute protocol notifications and forwards them in an IP network protocol, with the structure the consumer expects. The second is time stamping. Constrained nodes rarely have a reliable clock, so the time attached to each reading is the time of reception at the gateway. The third is memory: the gateway has permanent power and storage, and can therefore keep readings while the IP network is unavailable. The fourth is supervision of the connections, which includes detecting the loss of a node and restoring contact.

There are also less comfortable consequences. The gateway is a single point of failure for all the nodes it serves, and it is the security boundary between two networks of very different nature. Its radio range defines the area covered, which forces careful thought about its physical location.

4. Layered architecture

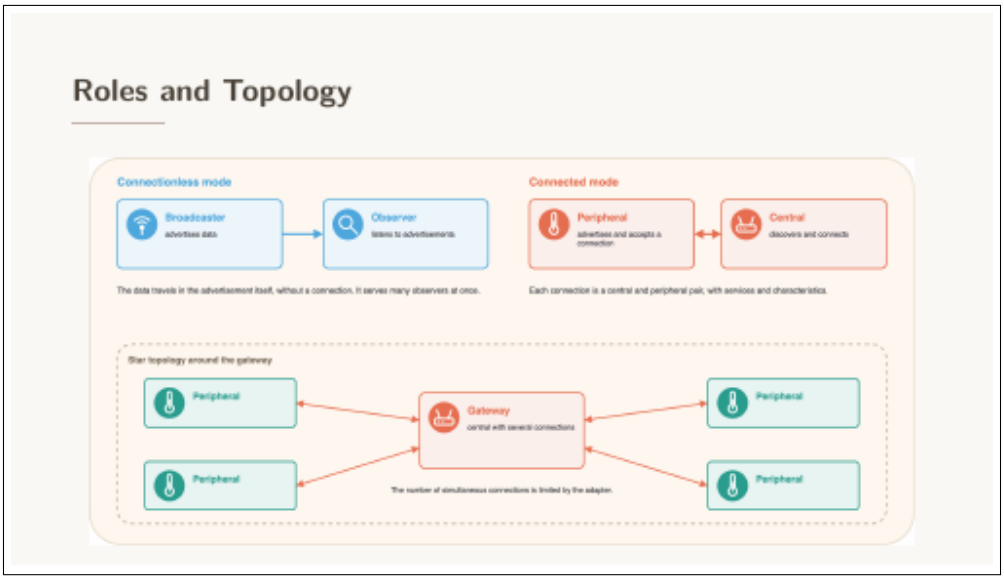


The IoT reference model used in Lecture 01 organises a system in levels, from the devices up to the applications. The diagram shows a reduced version with five layers. At the bottom are the BLE devices, with constrained sensors and actuators. Above is the connectivity and local processing layer, where the gateway lives. Next comes the IP network, which can be local, corporate or the Internet. Then comes storage and middleware, where databases and message brokers are found. At the top are the applications, which present, analyse and decide.

The gateway occupies a particular place. It belongs to the connectivity layer, because it bridges technologies, but it also performs local processing, because it filters, aggregates and stores before sending. This dual nature explains why the requirements of an IoT system end up concentrated in it.

The right hand column of the diagram lists the five questions that the rest of the lecture develops. Latency asks how long a datum takes to reach the consumer. Scale asks how many nodes a gateway supports. Reliability asks what happens when a connection fails. Security asks who may connect and who may read. Administration asks how to operate equipment with no screen or keyboard. Each of these questions has a concrete technical answer in the BLE mechanisms and in the gateway's programming choices.

5. Roles and topology in BLE



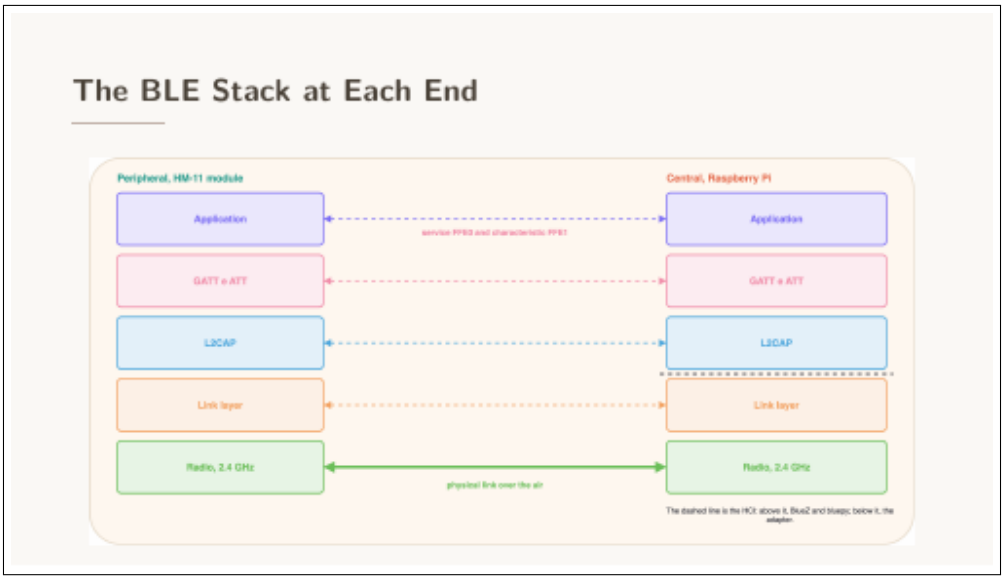
The Generic Access Profile, GAP, defines the roles a device can have. There are two modes of operation.

In connectionless mode there are the broadcaster and the observer. The broadcaster emits advertising packets with data and does not accept connections. The observer listens to those advertisements and takes the information from them. It is a useful mode when the same datum interests many receivers and no two way conversation is needed. Capacity is limited, because the data has to fit in the advertising payload, and delivery is not acknowledged.

In connected mode there are the peripheral and the central. The peripheral advertises and accepts a connection request. The central listens, chooses a peripheral and establishes the connection. From then on there is a two way channel, with services and characteristics, and with negotiated parameters. A gateway that collects sensor data uses this mode and behaves as the central.

The resulting topology is a star around the gateway. The central can keep several connections at the same time, but the number is limited by the adapter and the controller, and each connection consumes radio time. In the opposite direction, a peripheral such as the HM-11 module accepts a single central at a time. This asymmetry has practical effects: while the gateway is connected the node stops advertising, and no other device finds it. There is also a mesh technology on top of BLE, with a different topology, but it is not part of this lecture.

6. The BLE stack and where each layer runs



The BLE stack is split into two parts separated by a standard interface, the HCI. Below the HCI is the controller, which includes the radio and the link layer. Above is the host, which includes L2CAP, the attribute protocol, the generic attribute profile and the security manager. At the top is the application.

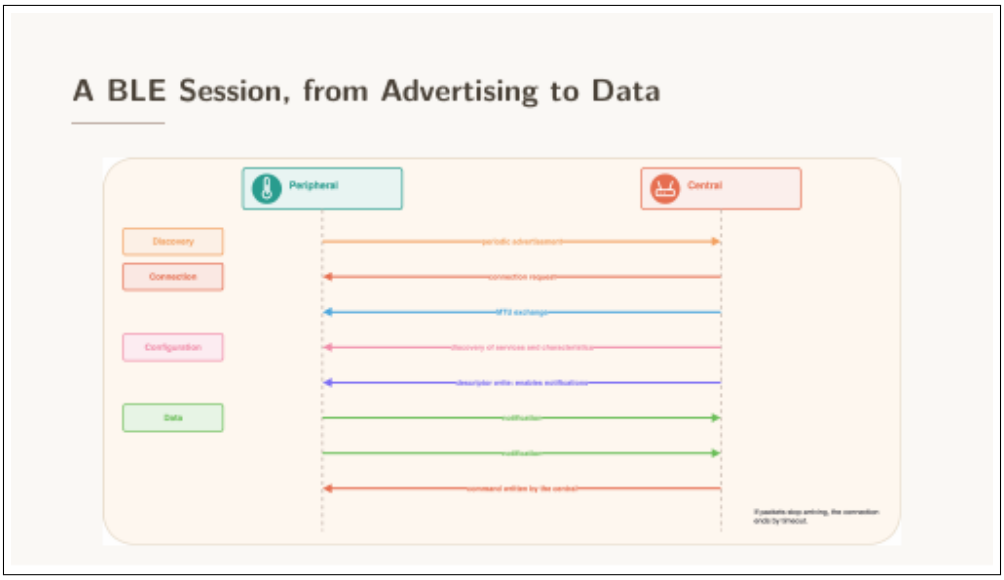
The controller handles what is time critical. It chooses the channels, applies frequency hopping, forms the packets, checks integrity and retransmits packets that were not acknowledged. These retransmissions happen entirely inside the controller, and so they are not visible at the HCI level.

The host handles the meaning of the data. The attribute protocol defines the server and its attributes. The generic attribute profile organises those attributes into services and characteristics. The security manager handles pairing and encryption.

On the HM-11 peripheral the full stack runs inside the module. The Arduino knows nothing of this: it only sees a serial port. That is why the module is said to expose a transparent serial link.

On the Raspberry Pi the split is different. The controller is in the board's Bluetooth adapter, and the host is BlueZ, which runs in the operating system. The bluepy library adds a Python interface on top of BlueZ. This separation matters because it determines what can be observed: the btmon tool records the traffic at the HCI, shows advertisements, connections and notifications, but does not reveal the controller's internal retransmissions.

7. Anatomy of a BLE session



A complete session goes through four phases, and understanding their order avoids many programming mistakes.

In the discovery phase the peripheral emits periodic advertisements and the central listens. The central identifies the device by its address, its name or the advertising data.

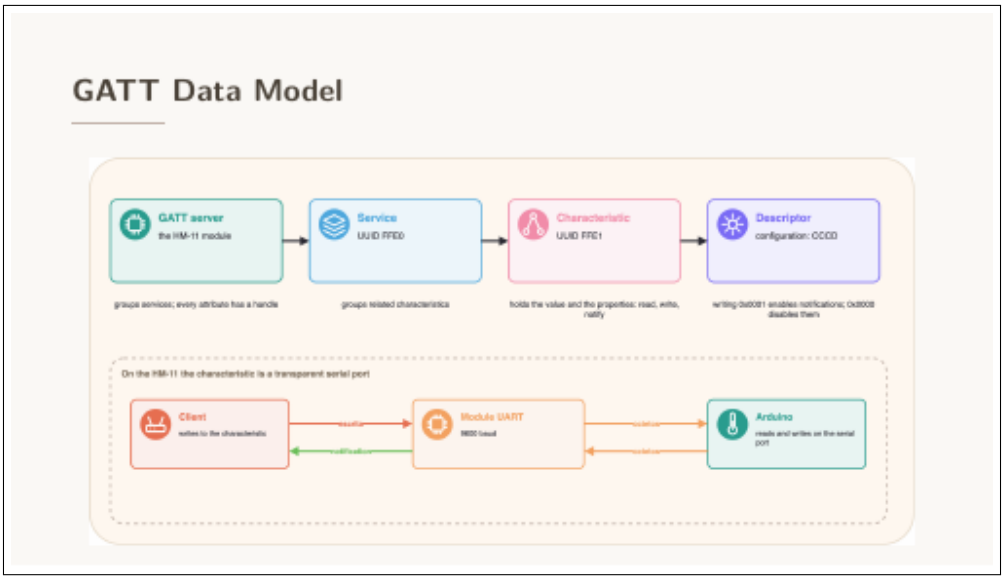
In the connection phase the central sends a connection request that includes the initial parameters, such as the interval between events. From that moment the peripheral stops advertising. Next, central and peripheral exchange the maximum size of the attribute protocol data unit, the MTU, which by default is 23 octets.

In the configuration phase the central discovers what the peripheral offers. It asks which services exist, then the characteristics of each service and finally the descriptors. Only after knowing the structure does it know which handle to write to. In this phase the client also writes to the characteristic's configuration descriptor to enable notifications.

In the data phase the peripheral sends notifications and the central can write commands. The phase ends when one side closes the connection or when the maximum time without packets, the supervision timeout, runs out.

A practical consequence is that the configuration descriptor is not enabled automatically. Whoever forgets to write it connects, discovers everything, and never receives a notification. Another consequence is that notifications do not survive a new connection, so a gateway that reconnects has to repeat the configuration phase.

8. The GATT data model



The attribute protocol defines that a server exposes a table of attributes. Each attribute has a handle, which is a 16 bit number local to the server, a type identified by a UUID and a value. The generic attribute profile gives this table structure through a hierarchy of three levels.

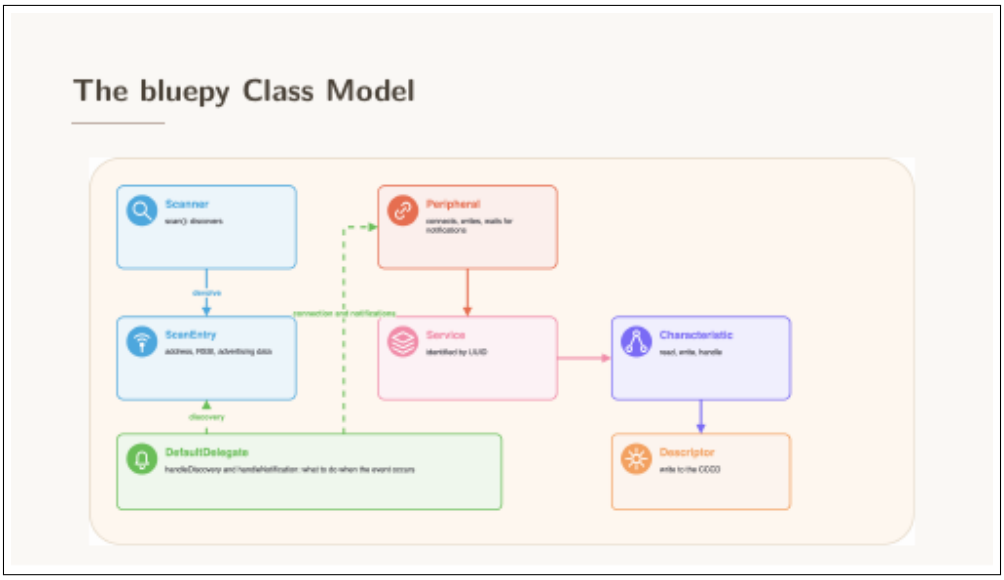
The service groups characteristics that are related to one another, such as those concerning the same sensor. The characteristic contains a value and a set of properties that indicate how it can be used: read, write with response, write without response, notify and indicate. The descriptor adds information about the characteristic, and the most important one for this lecture is the configuration descriptor, known by the acronym CCCD.

UUIDs can have 16 or 128 bits. The 16 bit ones are abbreviations of 128 bit UUIDs built on a base defined by Bluetooth. The HM-11 module uses the service with short UUID FFE0 and the characteristic FFE1.

The CCCD has UUID 0x2902 and holds two bits. The first enables notifications and the second enables indications. Writing the value 0x0001 enables notifications and writing 0x0000 disables them. In many simple servers the descriptor takes the handle right after the characteristic's, and the exercise code relies on that shortcut. A more robust implementation discovers the descriptors instead of assuming the position.

On the HM-11 the characteristic FFE1 works as a transparent serial port, as the lower part of the diagram shows. The octets the client writes to the characteristic leave through the module's serial port, at 9600 baud, and reach the Arduino. The octets the Arduino writes to the serial port become the value of the characteristic and are sent in notifications. The module thus translates a stream of octets into a set of attribute packets.

9. The Raspberry Pi as a BLE client: bluepy classes



The bluepy library gives access to BlueZ in Python and organises the operations in a few classes, which correspond to the phases of the session.

The Scanner class handles discovery. The scan method takes a duration and returns a list of entries. Each entry, of the ScanEntry class, holds the address, the address type, the RSSI and the advertising data, from which the device name, for example, can be extracted.

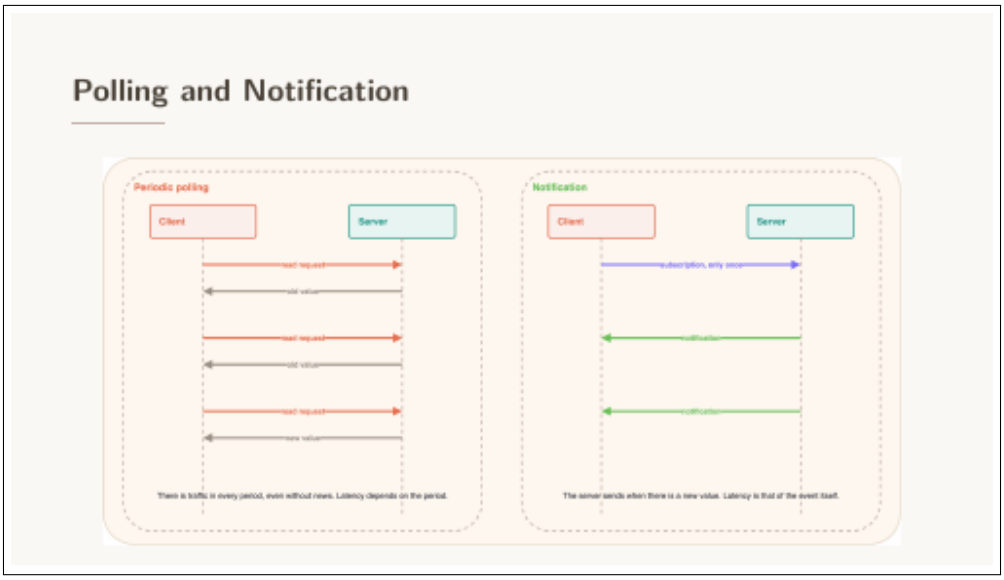
The Peripheral class represents a connection to a device. The constructor takes the address and blocks until the connection is established or fails. From the object one obtains the services and characteristics, and writes to handles, for example to enable notifications. The method that waits for notifications blocks until a notification arrives or the time runs out.

The DefaultDelegate class handles events. The programmer defines a subclass and implements the method that the library calls when something happens: the discovery of a device or the arrival of a notification. This pattern is called delegation and it inverts control. The application does not decide when the event occurs, it only declares what to do when it does.

Inversion of control is a central pattern in IoT. It reappears in message broker clients, in which the application registers a function that is called when a message arrives on a subscribed topic.

There are two limitations that shape the design of a gateway. The library is synchronous: while waiting for notifications, the program does nothing else on the same thread of execution. And loss of connection shows up as an exception, which the program has to handle. These two characteristics come up again when dealing with several nodes and with failure recovery.

10. Polling and notification



A client can obtain data from a characteristic in two ways.

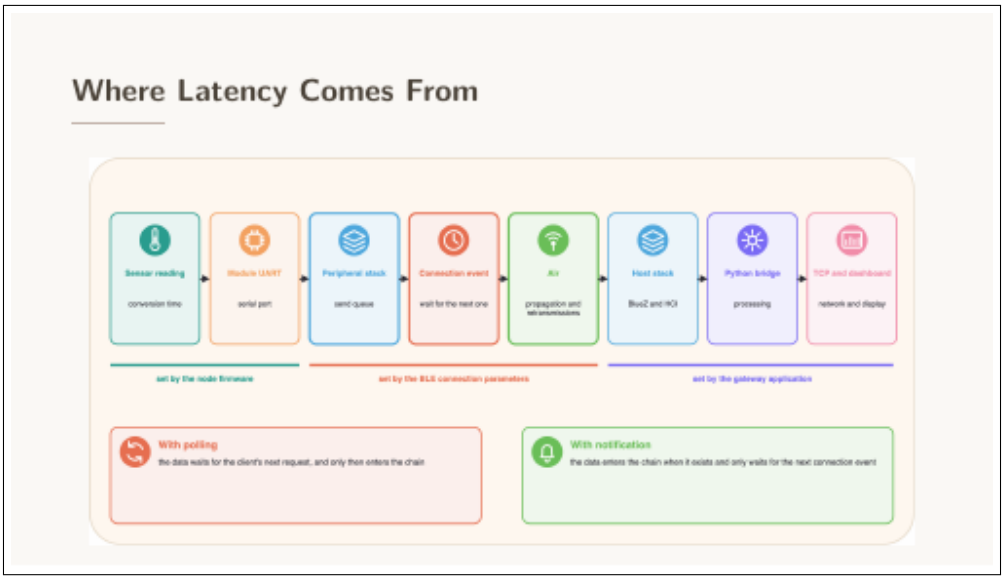
In polling, the client sends a read request and the server answers with the current value. It repeats the request at regular intervals. Each sample costs two attribute packets, and the traffic exists whether the value has changed or not. The server answers every request, even those that return a value the client already knows. Latency is bounded by the polling period, because a change is only seen at the next request.

In notification, the client subscribes once, by writing to the CCCD, and the server sends a packet whenever it has a new value. Each sample costs one packet, and there is traffic only when there is data. Latency is that of the event itself, plus only the wait for the transmission opportunity.

Notifications are not acknowledged by the attribute protocol. When confirmation at the application level is needed there is the indication, which requires a response from the client before the next one is sent, which reduces the pace and adds traffic. This lack of acknowledgement should not be confused with a lack of reliability. The link layer, below, acknowledges every packet and retransmits those that are lost as long as the supervision timeout allows. A notification can be lost only when the whole connection fails.

The choice between the two models depends on what the server can do. Notification requires the server to produce data on its own initiative, so the node firmware has to be written to send when it measures, and not only when asked. When the application tolerates the occasional loss of a sample, notification is almost always preferable.

11. Where latency comes from



The latency of a datum, from the moment the sensor produces it until it is displayed, is the sum of several stages, and the diagram lists them in order. Understanding the chain is what allows one to know where to act when the system seems slow.

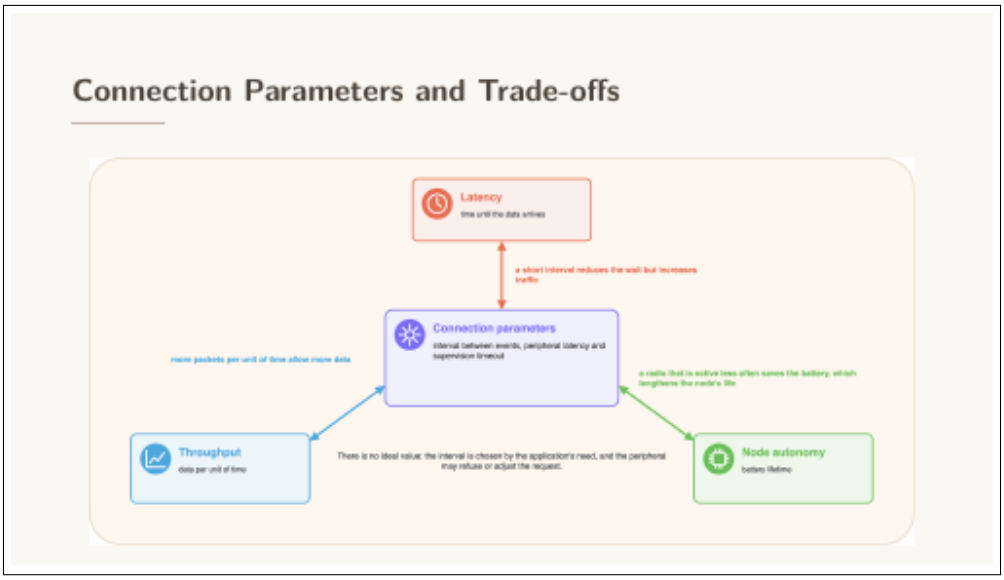
The first stage is the sensor reading, which can take the time of a conversion. The second is the transmission through the module's serial port, which takes the time to send all the octets of the message at the configured speed. The third is the send queue in the peripheral's stack. The fourth is the wait for the next connection event: the peripheral can only transmit at the instants when the central and it meet, and that interval is a connection parameter. The fifth is propagation through the air, which can include retransmissions when there is interference. Then come the processing in the host, the execution of the Python bridge, the transmission over TCP and the update of the dashboard.

The coloured bars under the stages show who decides each part. The node firmware decides the first ones, the BLE connection parameters decide the middle ones, and the gateway application decides the last ones. Only the middle part is determined by the protocol. In many real systems most of the latency is in the firmware or the application, and not in the radio.

The two lower cards compare the access models. With polling, the datum first waits for the client's next request and only then enters the chain. With notification, it enters the chain when it exists and only waits for the next connection event.

It is useful to distinguish latency from latency variation and from throughput. Latency is the delay of a datum. Latency variation, called jitter, is the difference between the delays of consecutive data, and it affects applications that depend on regularity. Throughput is the amount of data per unit of time. A system can have good throughput and bad latency, and the reverse.

12. Connection pace and trade-offs



Three parameters define the pace of a connection. The connection interval is the period between two events. The peripheral latency is the number of events it can skip when it has nothing to send. The supervision timeout is the time without valid packets after which the connection is declared ended.

The central proposes the values and the peripheral can ask for changes, which the central accepts or refuses. The effective values may therefore differ from those the module's documentation indicates.

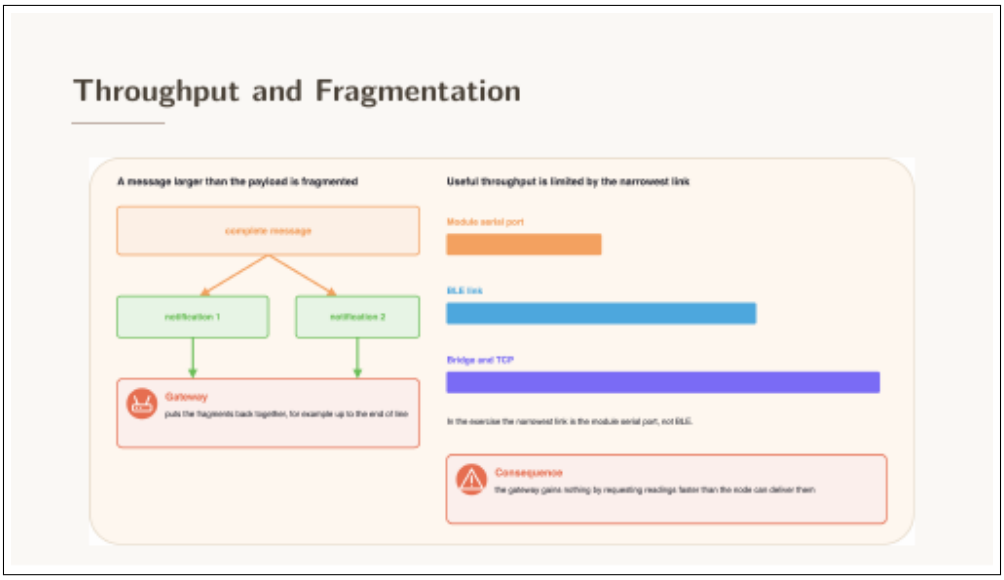
The diagram shows that these parameters put three properties in tension, and that no setting favours all of them. A short interval reduces the wait for the connection event and allows more packets per unit of time, which improves latency and throughput. But it forces the radio to be active more often, which reduces the autonomy of a battery powered node. A long interval does the opposite.

Peripheral latency allows the radio to be spared when the node has little to say. The cost is that a command sent by the central may wait several events before being heard, which weighs on systems with actuators.

The supervision timeout is a compromise between speed and stability. A short value detects a failure quickly, but may end connections that merely went through passing interference. A long value tolerates disturbances, but takes long to reveal a real failure. It has to be sufficiently larger than the interval, weighted by the peripheral latency, so that a few retransmissions fit inside it.

The choice is made from the application's requirement. A temperature sensor in a room tolerates long intervals. A manual command to an actuator demands fast responses.

13. Throughput and fragmentation



The default MTU of the attribute protocol is 23 octets. With the 3 octet header removed, each notification carries at most 20 octets of data. A larger message is split into several consecutive packets, and the receiver has to put it back together. Since BLE does not preserve the boundaries of the application message, it is usual to end each message with a delimiter, such as the end of line, and to join fragments until it is found.

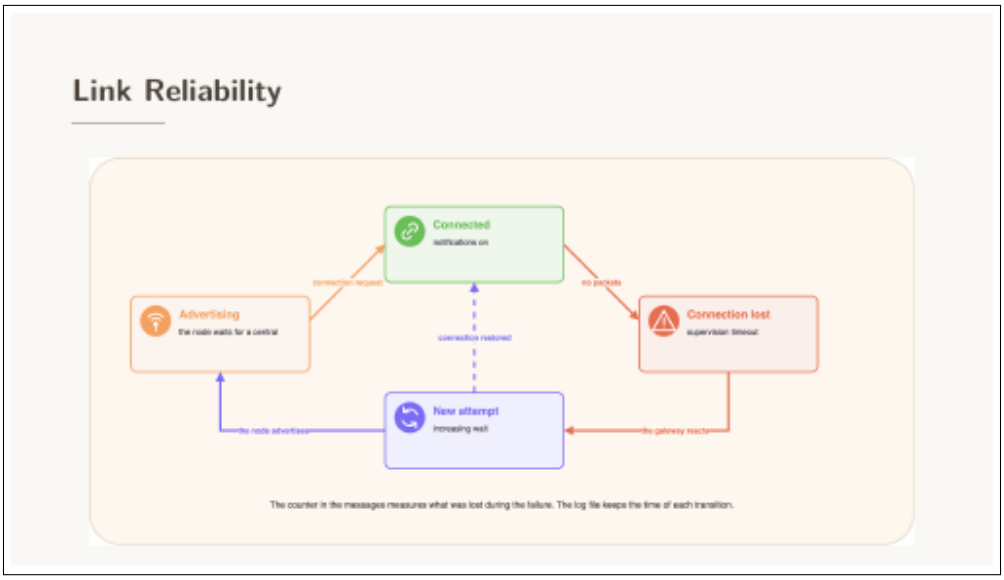
There are mechanisms to negotiate larger units, both at the MTU level and at the link packet level, but they depend on both sides supporting them. Simple modules such as the HM-11 stay at the default values.

Useful throughput is not the BLE throughput. It is the lowest of the throughputs of all the links the data goes through. The diagram compares three. The module's serial port, at 9600 baud, transfers fewer than a thousand octets per second. The BLE link has far greater capacity. The Python bridge and the local TCP connection are looser still.

The narrowest link, in this system, is the serial port, and not the radio. The practical consequence is that requesting readings faster than the node can deliver them improves nothing, it only fills queues. A well designed gateway knows the node's limit and sizes the pace accordingly.

The most effective strategy to improve application throughput is to reduce the size of the messages. A compact message, with few fields, fits in a single packet, avoids fragmentation and reduces the relative weight of the headers. The rich format, with identifier, unit and time, is built at the gateway and is not transmitted over the air.

14. Connection loss and recovery



A BLE connection can end by the will of one of the sides or by loss. In real systems loss is frequent: the node moves away, runs out of battery, is blocked by an obstacle or suffers prolonged interference.

The diagram describes the states the gateway goes through. With the connection established, notifications flow. When the peripheral stops answering, the connection is only considered lost at the end of the supervision timeout, so detection is never immediate. The library then signals the loss, in the case of bluepy through an exception, and the program moves to the retry phase.

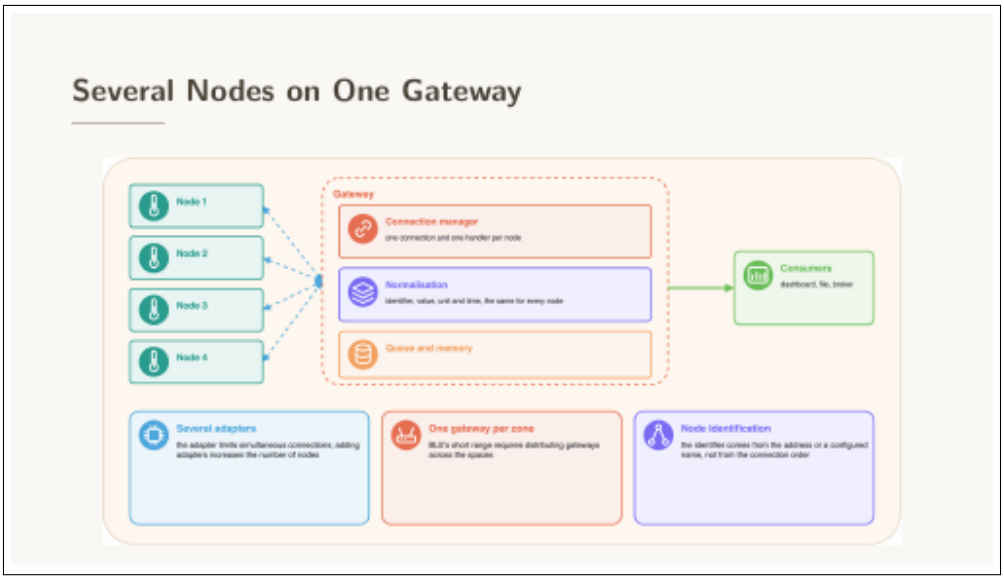
In the retry, the gateway tries to connect again. The strategy must avoid continuous attempts without a pause, which saturate the adapter and prevent other connections. The usual procedure is to progressively increase the interval between attempts up to a maximum, which is called increasing wait, or backoff. When the node advertises again, the connection is restored.

Restoring the connection does not restore the configuration. The configuration descriptor returns to its initial state, and the gateway has to repeat the write that enables notifications. It also has to resume any mode of operation that depends on the node, such as spontaneous sending.

There are two problems that reconnection does not solve by itself. The first is knowing how much data was lost during the failure. It is solved by numbering the messages at the node: a gap in the received sequence quantifies the losses. The second is detecting the node that appears connected but has stopped producing data. It is solved with an activity check in the application, which declares the node inactive when no readings arrive for a certain time.

Recording the time of each transition, loss and restoration, completes the information needed to assess the reliability of the system.

15. Several nodes, one gateway



A real gateway serves many nodes, and moving from one to several changes the design of the program.

The first question is concurrency. Each node has its own connection, its own parameters and its own data flow. Since bluepy is synchronous, a program with a single thread of execution can only wait for one node at a time. The usual solutions are one process or one thread per node, or an event loop that multiplexes the connections. In any case a connection management unit is useful, shown in the diagram, which creates, supervises and closes each connection individually.

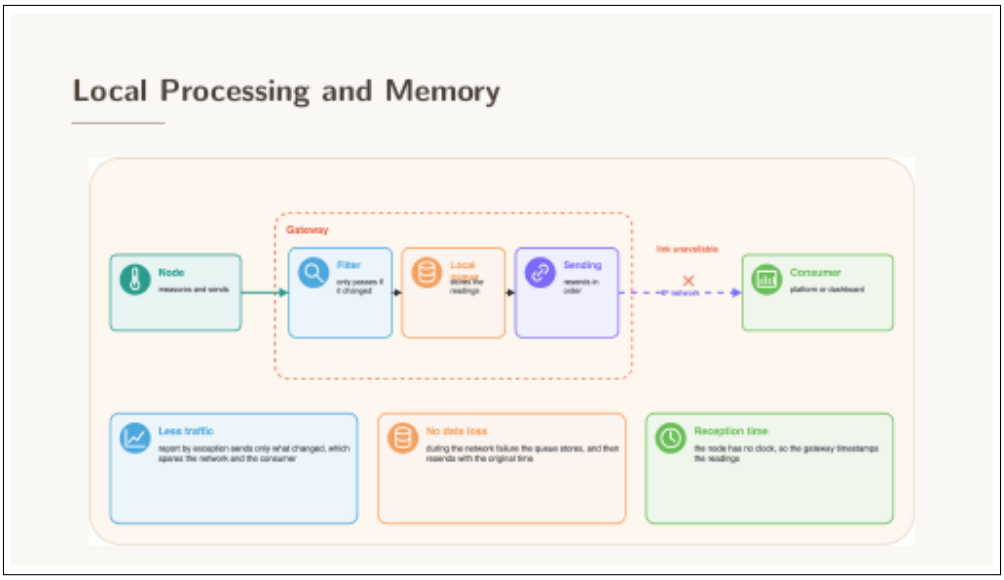
The second question is the adapter limit. The controller has to interleave the connection events of all the nodes without overlap. The more connections, and the shorter their intervals, the more contested the radio time becomes, and some connections start to suffer delay. The number of simultaneous connections an adapter supports is finite and depends on the controller. To go past the limit, adapters or gateways are added.

The third question is identification. Nodes have to be told apart from one another in a stable way. The identifier should not depend on the order in which connections are established, but on the device address or on a name assigned by configuration.

The fourth question is the standardisation of the data. The gateway converts the messages of each type of node to a common format, with identifier, value, unit and time. This normalisation decouples the consumers from the diversity of the nodes: a consumer that reads the common format does not need to know whether the value came from a BLE sensor or from another technology.

Finally, BLE range is short. An installation spread over several spaces needs one gateway per zone, which raises the problem of managing many gateways, their configuration and their monitoring.

16. Local processing and memory



Not everything the gateway receives should be forwarded immediately, and not everything needs to be sent. The gateway is the natural place for local decisions, because it is close to the nodes and has resources.

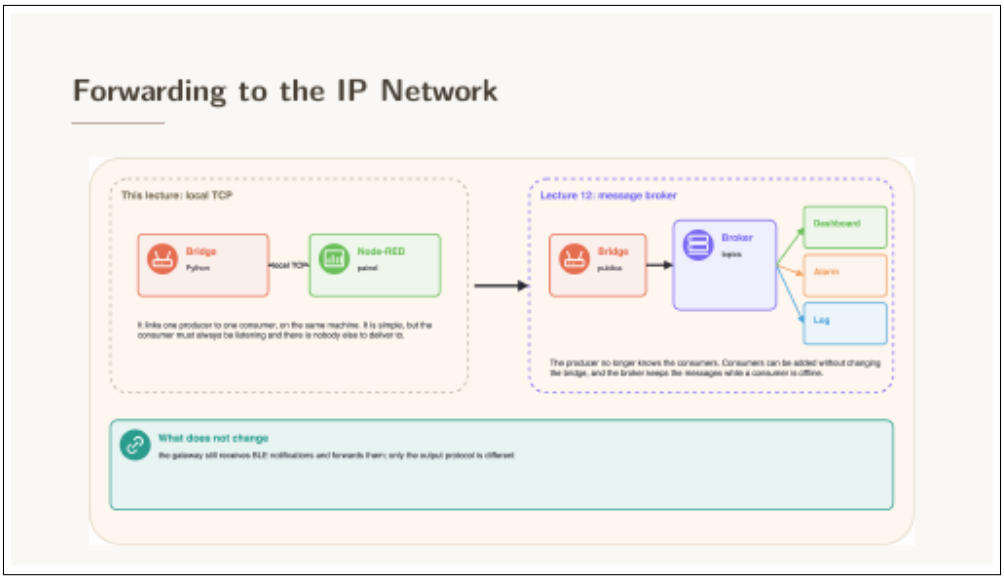
The filter reduces traffic. In report by exception, the gateway only forwards a reading when the value changes by more than a threshold. In systems that measure slowly varying quantities, this eliminates most messages without losing useful information. It reduces the load on the network, on the consumer and on storage.

The local queue gives resilience to network failures. When the connection to the IP network drops, the gateway does not lose the readings: it stores them and resends them when the network returns. The pattern is called store and forward. It has to solve some practical issues. The queue must keep the order of the readings. It must have a limited size, with a clear policy on what to discard when it fills. It should be able to be persistent, on disk, if the gateway can restart. And the resending may produce duplicates, which the consumer has to know how to handle.

The time has to be assigned at reception and preserved on resending. Since the node has no clock, it is the gateway that stamps the readings. If the time were assigned only at sending, a reading stored during a network failure would look recent, and time based analyses would be wrong.

This design has an additional advantage. The gateway can make decisions, such as raising a local alarm, even if the network or the remote platform is unavailable.

17. Forwarding to the IP network



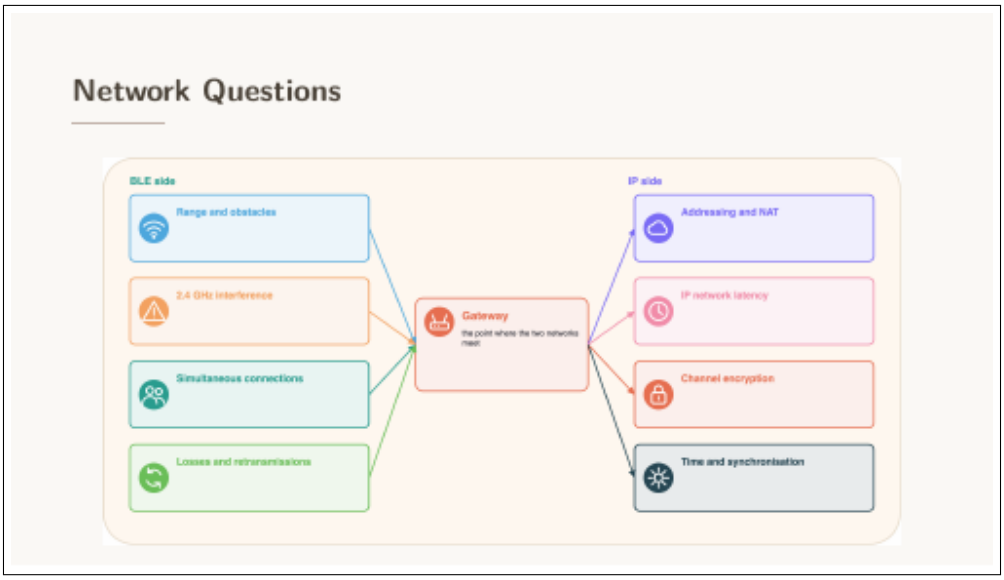
Once received and processed, the readings have to reach whoever uses them. The diagram compares two ways of doing so.

In the first, which is the one used in this lecture, the Python bridge connects over local TCP to Node-RED. Producer and consumer are on the same machine and communicate through a direct link. It is simple and needs no infrastructure. It has clear limitations. The consumer has to be always active and listening. The producer knows the consumer, so adding a second consumer means changing the bridge. And there is no intermediate memory: if the consumer is stopped, the data is lost.

In the second, which will be studied in Lecture 12, the bridge publishes the messages to a broker. The broker organises the messages by topics, and the consumers subscribe to the topics that interest them. The producer no longer knows the consumers. Dashboards, alarms and logs can be added without touching the bridge. The broker can retain messages while a consumer is offline. Producer and consumer are said to be decoupled in space, because they do not know each other, and in time, because they do not have to be active at the same time.

The structure of the gateway stays the same in both versions. It still receives BLE notifications, normalises them and forwards them. Only the protocol of the output interface changes. That is why the architecture of the gateway is studied before the application protocols and message middleware: what is learned now applies unchanged to those protocols.

18. Gateway network questions

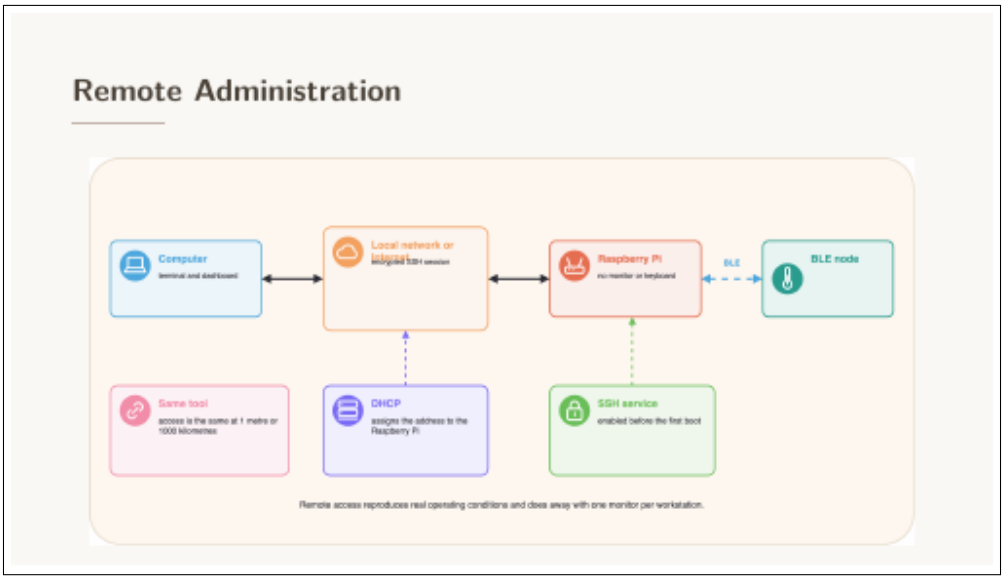


The gateway sits between two networks with different problems.

On the BLE side, range depends on power, antenna, obstacles and orientation. Walls, metals and human bodies attenuate the signal and make the RSSI vary, so it is not a reliable measure of distance. The 2.4 GHz band is shared with Wi-Fi and other equipment. BLE's adaptive frequency hopping avoids busy channels, but interference still causes retransmissions and degrades the pace. On a Raspberry Pi there is a further particularity: the Wi-Fi radio and Bluetooth share the same board and can compete for the same spectrum. The number of simultaneous connections is limited, and the rate of losses and retransmissions conditions the effective capacity.

On the IP side, addressing deserves attention. A gateway behind an address translation device can start connections outwards, but does not receive connections from outside without additional configuration. The gateway's own address can change if it is assigned by DHCP. The latency and losses of the wide area network can be much greater than those of the local network, and the design must tolerate them, which reinforces the role of the local queue. The channel to the network has to be encrypted, because the data crosses networks we do not control. And the gateway's time has to be correct, because it is the one that stamps the readings. A wrong clock corrupts all the data and should be synchronised with a network time service.

19. Remote administration



A gateway installed in the field rarely has a screen and keyboard. All configuration, diagnosis and updating is done remotely, and this mode of operation is called headless.

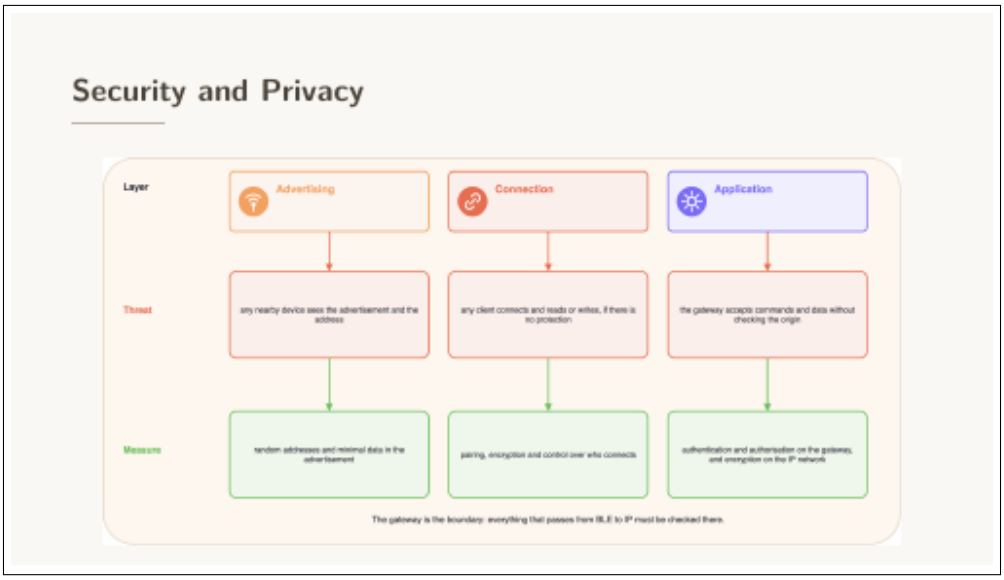
The usual instrument is SSH, which opens an encrypted command line session over the network. The session works the same way for a gateway in the same room or thousands of kilometres away, as long as there is a network path to it.

To reach the gateway one needs to know its address. On a local network, the DHCP server assigns it, and it is found by consulting the server's client list, by the machine name or by scanning the network. A long lived installation benefits from a reserved address or a stable name.

The SSH service has to be active. On the Raspberry Pi, in the versions used in the course, it is enabled by creating an empty file named ssh on the card's boot partition before the first boot. In other versions, the option is chosen while preparing the card.

A few precautions complete the practice. For a gateway reachable outside the local network, key authentication is safer than a password, and access through a virtual private network is preferable to opening ports directly. It is worth checking remotely the state of the Bluetooth adapter, which is the first point of failure of a BLE gateway. And keeping a log of the gateway's events makes it possible to diagnose remotely what went wrong.

20. Security and privacy



The security of a BLE system with a gateway is decided in layers, and the diagram relates each one to its threat and its measure.

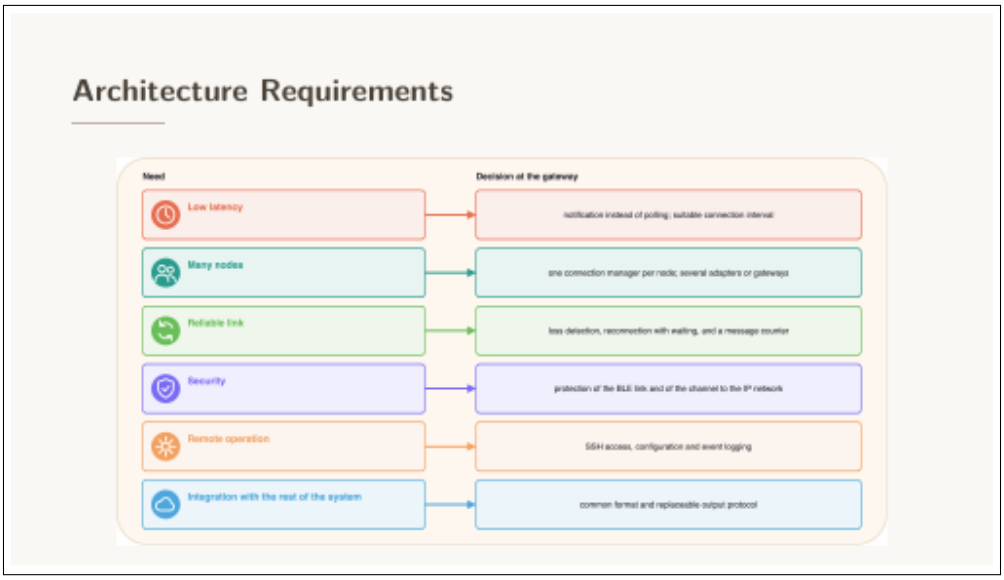
At the advertising layer, any nearby device sees the packets and the address. This allows information about the devices in a space to be collected without any authorisation, and allows a person to be followed through a device that always uses the same address. The privacy measure is to use random addresses that change over time and to keep the advertising data to a minimum. Addresses can be public, assigned by the manufacturer, or random, and only renewable random ones protect against tracking.

At the connection layer, the threat is that any client connects, reads and writes. A module such as the HM-11, in its default configuration, accepts any central and does not encrypt the traffic. The measure is to use pairing and link encryption, and to limit who can connect. BLE has pairing and secure connection mechanisms that the module may or may not support, and that support should be checked before choosing the module.

At the application layer, the gateway should not accept data or commands without checking the origin. It should validate the format of what it receives, limit the rate of commands and authenticate whoever gives it orders. For the IP network, the channel should be encrypted, and access credentials should not be written in the code, but in environment variables or protected files.

The gateway is the boundary between two networks and their control point. What passes from BLE to IP should be checked there, and what comes down from the IP network to the nodes should be authorised there.

21. Architecture requirements and decisions

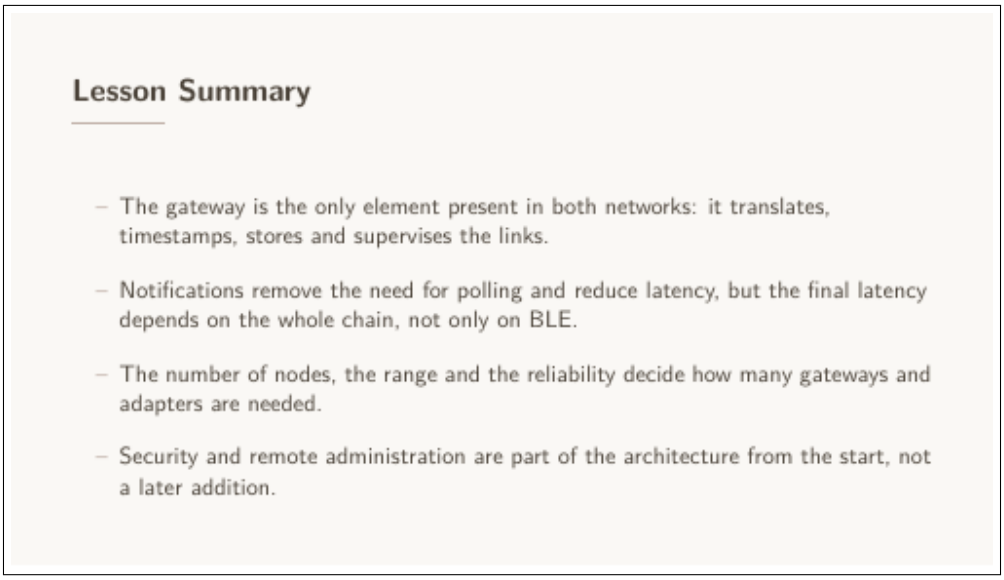


The last diagram summarises the lecture as a correspondence table between needs and decisions. Each row starts with a system need and ends in a decision taken at the gateway.

For low latency, the decision is to use notifications instead of polling and to choose a connection interval suited to the requirement. To support many nodes, the decision is to give the gateway a connection management unit with independent handling per node, and to use several adapters or gateways when the limit is reached. For reliability, the decision is to detect the loss, reconnect with increasing wait, repeat the configuration and number the messages to measure losses. For security, the decision is to protect the BLE link and the channel to the IP network and to check at the boundary what passes. For remote operation, the decision is to access by SSH, configure remotely and log events. For integration with the rest of the system, the decision is to adopt a common format and keep the output protocol replaceable.

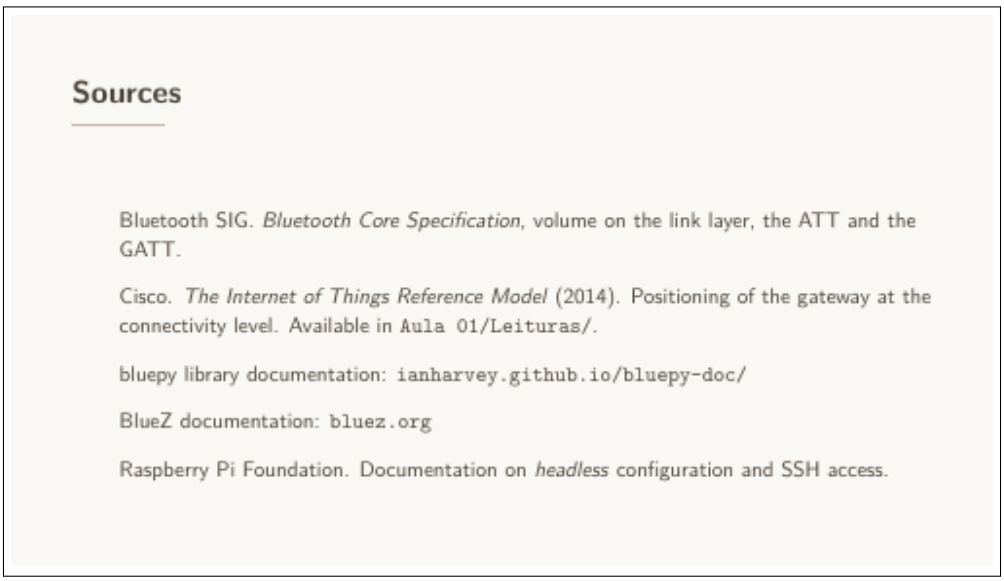
The idea that runs through the six rows is that the qualities of an IoT system result from architecture decisions, and not only from the choice of radio technology. BLE provides the mechanisms, but it is the design of the gateway that decides whether the system is fast, scalable, reliable and secure.

22. Lecture summary



The gateway links the BLE network, without IP, to the data network, and concentrates translation, memory, time and supervision. Notification reduces traffic and latency compared with polling, but the final latency is the sum of a chain of stages, of which only part belongs to BLE. Scale, reliability and security are architecture requirements that are solved at the gateway, through the management of several connections, reconnection with increasing wait and checking at the boundary. Forwarding over local TCP will be replaced by a message broker, without changing the structure of the gateway. These themes return in the following lectures, with other access technologies and with the application protocols.

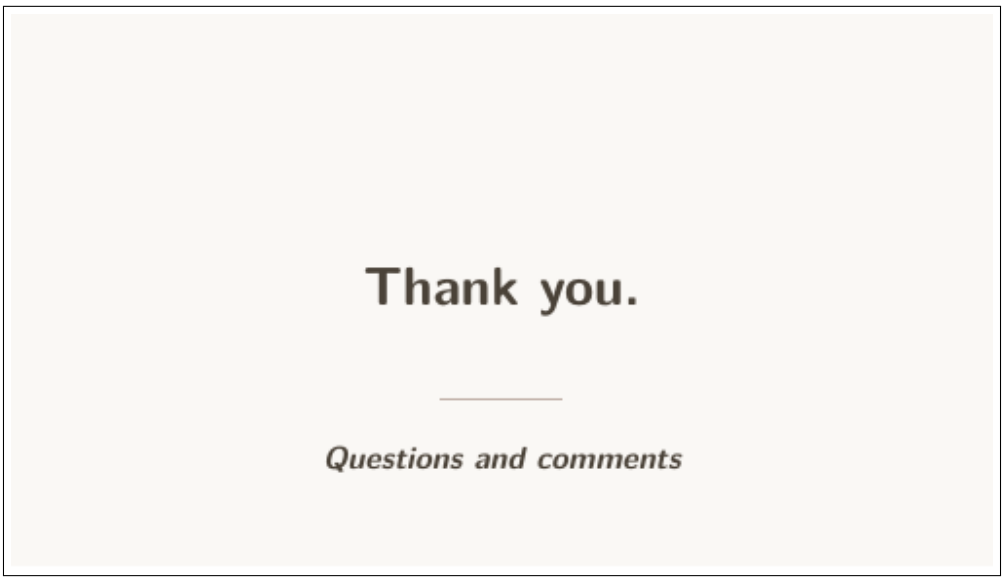
23. Sources



The Bluetooth core specification is the reference for the roles, the attribute protocol, the generic attribute profile and the link layer, and it is where the allowed ranges for each parameter are

confirmed. The Cisco IoT reference model, available in the readings of Lecture 01, frames the place of the gateway in the architecture. The bluepy library documentation describes the classes and methods used by the course programs, and the BlueZ documentation describes the stack that runs on the Raspberry Pi. The Raspberry Pi Foundation documentation covers headless configuration and SSH access. The lab guides of previous editions were the basis for the remote administration and BLE client part.

24. Closing



The essential point is that a gateway is not just an adapter between technologies. It is the place where latency, scale, reliability, security and operation are decided, and its design conditions everything above it.